

从环境搭建到结果采集，一套完成的python爬虫搭建 | 附源代码

2017-11-17 [BestSDK](#)

最近业余在做一个基于.NET Core的搜索项目，奈何基层代码写好了，没有看起来很华丽的数据供测试。很巧的也是博客搜索，于是乎想到了博客园。C#也能做做页面数据抓取的，不过在博客园看到的大部分都是python实现，所以就临时想了一下看看python到底是什么东东，不看基础语法，不看语言功能，直接上代码，哪里不会搜哪里。代码完成总共用时大概4个小时，其中搭建环境加安装BeautifulSoup大概1个小时。解析HTML用时间最多了，边看demo边解析，大概2个小时，剩下的时间就是调试加保存数据了。

环境搭建

既然用python，那么自然少不了语言环境。于是乎到官网下载了3.5版本的。安装完之后，随机选择了一个编辑器叫PyCharm，话说python编辑器还真挺多的。由于本人是小白，所以安装事项不在过多赘述。

```
Microsoft Windows [版本 6.3.9600]
(c) 2013 Microsoft Corporation. 保留所有权利。

C:\Users\Administrator>python
Python 3.5.3 (v3.5.3:1880cb95a742, Jan 16 2017, 15:51:26) [MSC v.1900 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> _
```

建好项目，打开编辑器，直接开工。本来之前用C#写的时候，大体思路就是获取网页内容，然后正则匹配。后来发现网上的帖子也很多。不过在搜索过程中发现，不建议用正则来匹配HTML。有正好我的正则不太好，所以我就搜了一下HTML解析工具，果不其然，人家都做好了，直接拿来用吧。没错就是这个东东：BeautifulSoup。安装也很简单，不过中间出了个小插曲，就是bs4没有。继续搜，然后需要用pip安装一下就好了。（当然我并不知道ps4和pip是什么鬼）

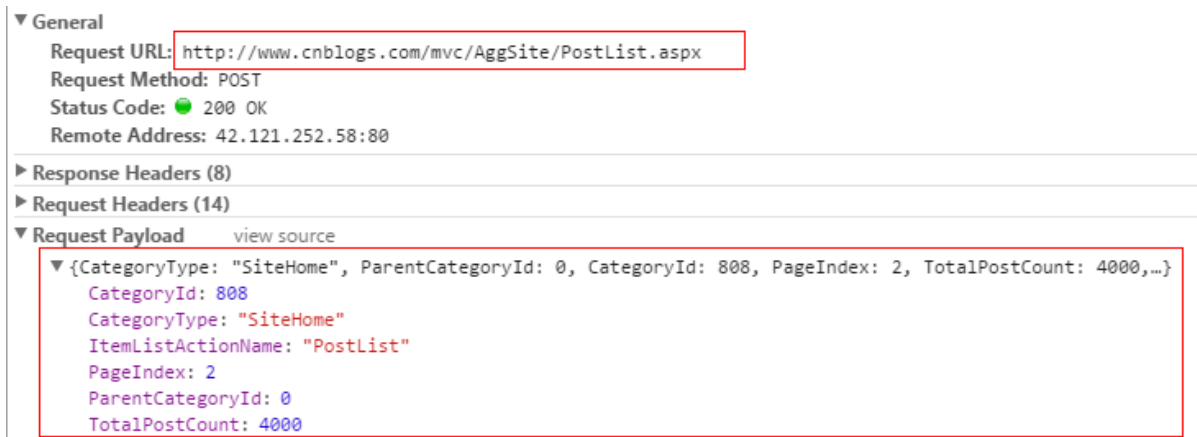
```
C:\Windows\system32\cmd.exe - python
No matching distribution found for sklean

D:\Python\Scripts>pip install bs4
Collecting bs4
  Downloading bs4-0.0.1.tar.gz
Collecting beautifulsoup4 (from bs4)
  Downloading beautifulsoup4-4.5.3-py3-none-any.whl (85kB)
    48% |#####| 40kB 174kB/s eta 0:00
    60% |#####| 51kB 192kB/s eta
    72% |#####| 61kB 231kB/s e
    84% |#####| 71kB 241kB
    96% |#####| 81kB 2
   100% |#####| 92kB
294kB/s
Installing collected packages: beautifulsoup4, bs4
  Running setup.py install for bs4 ... done
Successfully installed beautifulsoup4-4.5.3 bs4-0.0.1

D:\Python\Scripts>python
Python 3.5.3 (v3.5.3:1880cb95a742, Jan 16 2017, 15:51:26) [MSC v.1900 32 bit (Intel)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> from bs4 import BeautifulSoup
>>> _
```

思路分析

博客吗，我当然就对准了博客园，于是乎，进入博客园首页，查看请求。



发送请求

当然我不知道python是怎么进行网络请求的，其中还有什么2.0和3.0的不同，中间曲曲折折了不少，最终还是写出了最简单的一段请求代码。

```
1 import urllib.parse
2 import urllib.request
3
4 # params CategoryId=808 CategoryType=SiteHome ItemListActionName=PostList
5 def getHtml(url, values):
6     user_agent = 'Mozilla/5.0 (Windows NT 6.3; WOW64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/41.0.2272.134 Safari/537.36'
7     headers = {'User-Agent': user_agent}
8     data = urllib.parse.urlencode(values)
9     response_result = urllib.request.urlopen(url + '?' + data).read()
10    html = response_result.decode('utf-8')
11    return html
12
13 # 获取数据
14 def requestCnblogs(index):
15     print('请求数据')
16     url = 'http://www.cnblogs.com/mvc/AggSite/PostList.aspx'
17     value = {
18         'CategoryId': 808,
19         'CategoryType': 'SiteHome',
20         'ItemListActionName': 'PostList',
21         'PageIndex': index,
22         'ParentCategoryId': 0,
23         'TotalPostCount': 4000
24     }
25     result = getHtml(url, value)
26     return result
```

其实博客园这个请求还是挺标准的，哈哈正好适合抓取。因为他返回的就是一段html。（如果返回json那不是更好。。。。）

数据解析

上文已经提到了，用到的是BeautifulSoup，好处就是不用自己写正则，只要根据他的语法来写就好了，在多次的测试之后终于完成了数据的解析。先上一段HTML。然后在对应下面的代码，也许看起来更轻松一些。

```

1 <div class="post_item">
2   <div class="digg">
3     <div class="diggit" onclick="DiggPost('hyper-xl',6417741,281238,1)"
4       <span class="diggnum" id="digg_count_6417741">1</span>
5     </div>
6     <div class="clear"></div>
7     <div id="digg_tip_6417741" class="digg_tip"></div>
8   </div>
9   <div class="post_item_body">
10    <h3><a class="titlelnk" href="http://www.cnblogs.com/hyper-xl/p/64
11
12
13    <p class="post_item_summary">
14      <a href="http://www.cnblogs.com/hyper-xl/" target="_blank">
15        
25      <a href="http://www.cnblogs.com/hyper-xl/" class="lightblue">亲
26      发布于 2017-02-19 23:07
27      <span class="article_comment">
28        <a href="http://www.cnblogs.com/hyper-xl/p/6417741.html#co
29        评论(0)
30      </a>
31    </span>
32    <span class="article_view">
33      <a href="http://www.cnblogs.com/hyper-xl/p/6417741.html" c
34      阅读
35
36      (138)
37    </a>
38  </span>
39 </div>
40 </div>
41 <div class="clear"></div>
42 </div>

```

通过上文的HTML代码可以看到几点。首先每一条数据都在 `div(class="post_item")` 下。然后 `div("post_item_body")` 下有用户信息，标题，链接，简介等信息。逐一根据样式解析即可。代码如下：

```

1 from bs4 import BeautifulSoup
2 import request
3 import re
4
5 #解析最外层
6 def blogParser(index):
7
8     cnblogs = request.requestCnblogs(index)
9     soup = BeautifulSoup(cnblogs, 'html.parser')
10    all_div = soup.find_all('div', attrs={'class': 'post_item_body'}, limit=
11
12    blogs = []
13    #循环div获取详细信息
14    for item in all_div:
15        blog = analyzeBlog(item)
16        blogs.append(blog)
17
18    return blogs
19
20 #解析每一条数据
21 def analyzeBlog(item):
22     result = {}
23     a_title = find_all(item, 'a', 'titlelnk')
24     if a_title is not None:
25         # 博客标题
26         result["title"] = a_title[0].string
27         # 博客链接
28         result["href"] = a_title[0]['href']
29     p_summary = find_all(item, 'p', 'post_item_summary')
30     if p_summary is not None:
31         # 简介
32         result["summary"] = p_summary[0].text
33     footers = find_all(item, 'div', 'post_item_foot')
34     footer = footers[0]
35     # 作者
36     result["author"] = footer.a.string
37     # 作者url
38     result["author_url"] = footer.a['href']
39     str = footer.text
40     time = re.findall(r"发布于 .+? .+? ", str)
41     result["create_time"] = time[0].replace('发布于 ', '')
42
43     comment_str = find_all(footer, 'span', 'article_comment')[0].a.string
44     result["comment_num"] = re.search(r'\d+', comment_str).group()
45
46     view_str = find_all(footer, 'span', 'article_view')[0].a.string
47     result["view_num"] = re.search(r'\d+', view_str).group()
48
49     return result
50
51 def find_all(item, attr, c):
52     return item.find_all(attr, attrs={'class': c}, limit=1)

```

上边一堆代码下来，着实花费了我不少时间，边写边调试，边百度~~不过还好最终还是出来了。等数据都整理好之后，然后我把它保存到了txt文件里面，以供其他语言来处理。本来想写个put直接put到ElasticSearch中，奈何没成功。后边在试吧，毕竟我的重点只是导出数据，不在抓取这里。


```

1 import match
2 import os
3 import datetime
4 import json
5
6 def writeToTxt(list_name,file_path):
7     try:
8         #这里直接write item 即可，不要自己给序列化在写入，会导致json格式不正确的问题
9         fp = open(file_path,"w+",encoding='utf-8')
10        l = len(list_name)
11        i = 0
12        fp.write('[')
13        for item in list_name:
14            fp.write(item)
15            if i<l-1:
16                fp.write(',\n')
17                i += 1
18        fp.write(']')
19        fp.close()
20    except IOError:
21        print("fail to open file")
22
23 #def getStr(item):
24 #    return json.dumps(item).replace('\','\"')+','+\n'
25
26 def saveBlogs():
27     for i in range(1,2):
28         print('request for '+str(i)+'...')
29         blogs = match.blogParser(i,5)
30         #保存到文件
31         path = createFile()
32         writeToTxt(blogs,path+'/blog_'+ str(i) + '.json')
33         print('第'+ str(i) +'页已经完成')
34     return 'success'
35
36 def createFile():
37     date = datetime.datetime.now().strftime('%Y-%m-%d')
38     path = '/' + date
39     if os.path.exists(path):
40         return path
41     else:
42         os.mkdir(path)
43         return path
44
45 result = saveBlogs()
46 print(result)

```

上边呢，我取了一百页的数据，也就是大概2000条做测试。

成果验收

废了好大劲终于写完那些代码之后呢，就可以享受胜利的果实了，虽然是初学者，代码写的很渣，这参考一下，那参考一下，不过还是有些收获的。运行效果如下：

```

请求数据
第57页已经完成
请求数据
第58页已经完成
请求数据
第59页已经完成
请求数据
第60页已经完成
请求数据
第61页已经完成
请求数据
第62页已经完成
请求数据
第63页已经完成

```

生成的文件：

blog_1.txt	2017/2/20 22:02	文本文档	13 KB
blog_2.txt	2017/2/20 22:02	文本文档	12 KB
blog_3.txt	2017/2/20 22:02	文本文档	13 KB
blog_4.txt	2017/2/20 22:02	文本文档	13 KB
blog_5.txt	2017/2/20 22:02	文本文档	14 KB
blog_6.txt	2017/2/20 22:02	文本文档	14 KB
blog_7.txt	2017/2/20 22:02	文本文档	13 KB
blog_8.txt	2017/2/20 22:02	文本文档	13 KB
blog_9.txt	2017/2/20 22:02	文本文档	13 KB
blog_10.txt	2017/2/20 22:02	文本文档	14 KB
blog_11.txt	2017/2/20 22:02	文本文档	13 KB
blog_12.txt	2017/2/20 22:02	文本文档	13 KB
blog_13.txt	2017/2/20 22:02	文本文档	13 KB
blog_14.txt	2017/2/20 22:02	文本文档	13 KB
blog_15.txt	2017/2/20 22:02	文本文档	14 KB
blog_16.txt	2017/2/20 22:02	文本文档	13 KB
blog_17.txt	2017/2/20 22:02	文本文档	13 KB
blog_18.txt	2017/2/20 22:02	文本文档	13 KB
blog_19.txt	2017/2/20 22:02	文本文档	13 KB
blog_20.txt	2017/2/20 22:02	文本文档	13 KB
blog_21.txt	2017/2/20 22:02	文本文档	13 KB

文件内容：

```
blog_1.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
{"href": "http://www.cnblogs.com/demonxian3/p/6421788.html", "summary": "\n
1.搭建lamp环境yum -y install httpd php php-mysql mysql mysql-server 2启动
服务 3.设置服务开机自动启动 4.上传UCENTER 并解压unzip
UCenter_1.6.0_SC_UTF8.zip 5.解压后会在当前目录生成一个readm...\r\n
",
"author": "Demon贤3", "comment_num": "0", "title": "centos安装UCenter 和
UCenter_Home", "view_num": "2", "author_url":
"http://www.cnblogs.com/demonxian3/", "create_time": "2017-02-20 22:03 "}
{"href": "http://www.cnblogs.com/saper/p/6421687.html", "summary": "\n 上
一次的《入职一周》是在2012-04-22写的,写的是入职到立达信的情景,不过这次是为
入职到新公司而写。很荣幸我能入职到这家公司,如果没有领导的青睐,也许我现在
有大概率还在家呢。今年SAP市场好像不是很好,所以自己算是很幸运的。新公司距离
我住的地方也就公交车8分钟,加上走路和等车的时间,大概得...\r\n
",
"author": "梦心", "comment_num": "0", "title": "入职一周", "view_num": "4",
"author_url": "http://www.cnblogs.com/saper/", "create_time": "2017-02-20
22:03 "}
{"href": "http://www.cnblogs.com/xumengxuan/p/6421744.html", "summary": "\n
我们需要搭建的这个简易的论坛主要的功能有:注册、登录、发布主题、回复主题。
下面我们来一步步地讲解这个系统是如何实现的。...\r\n
",
"author":
"wenzi0_0", "comment_num": "0", "title": "从0到1学习node(七)之express搭建简
易论坛", "view_num": "12", "author_url":
"http://www.cnblogs.com/xumengxuan/", "create_time": "2017-02-20 21:54 "}
```

总结

一个简单的抓取程序就写完了，python还真是TM的好用。以后有空再研究研究吧。代码行数算上空行和注释总共 100 （50+25+25） 行。

智选SDK

聪明的产品经理和开发者都在这儿



微信号：bestsdk

[阅读原文](#)